

Introduction to Unix shell

Part 4

Mirco Marchetti, Riccardo Lancellotti

SECloud research group

University of Modena and Reggio Emilia

AY 2025/26

Advanced Scripting and OS Interaction

- Functions allow you to group commands into reusable blocks of code
- Functions can accept parameters and return values
- Basic syntax for defining a function:
 - `function_name() { <commands> }`
 - `function function_name { <commands> }`
- Both syntax are equivalent; the first is more common in modern Bash
- Functions must be defined before they are called
- Calling a function: `function_name [arguments]`

Function Definition Examples

```
#!/bin/bash
# Simple function with no parameters
greet() {
    echo "Hello, World!"
}
# Function with parameters
add() {
    local SUM=$(( $1 + $2 ))
    echo $SUM
}
# Function with return value
is_file() {
    if [ -f "$1" ]; then
        return 0 # success
    else
        return 1 # failure
    fi
}
```

```
# Calling the functions
greet
RESULT=$(add 5 3)
echo "Sum: $RESULT"
is_file "/etc/passwd" && echo "File exists"
```

- Function parameters are accessed using positional parameters
 - `$0`: The name of the script (not the function name)
 - `$1, $2, ...`: Function arguments
 - `$#`: Number of arguments passed to the function
 - `$@`: All arguments as separate words
 - `$*`: All arguments as a single string
- Variables declared inside a function are global by default
- Use `local` keyword to create local variables
 - `local VARIABLE="value"`: Creates a local variable
 - Local variables are only accessible within the function
- Functions can return values using the `return` statement
 - Return values are exit codes (0-255)
 - Use command substitution to capture output: `RESULT=$(function_name)`

Local Variables and Scope

```
#!/bin/bash
GLOBAL_VAR="I am global"

demonstrate_scope() {
    local LOCAL_VAR="I am local"
    GLOBAL_VAR="Modified globally"
    echo "Inside function:"
    echo "  LOCAL_VAR: $LOCAL_VAR"
    echo "  GLOBAL_VAR: $GLOBAL_VAR"
}

echo "Before function call:"
echo "  GLOBAL_VAR: $GLOBAL_VAR"

demonstrate_scope

echo "After function call:"
echo "  GLOBAL_VAR: $GLOBAL_VAR"
echo "  LOCAL_VAR: $LOCAL_VAR" # empty, local scope ended
```

Functions with Return Values

```
#!/bin/bash
# Function returns output via echo
calculate_area() {
    local WIDTH=$1
    local HEIGHT=$2
    local AREA=$((WIDTH * HEIGHT))
    echo $AREA
}

# Function returns status via return
check_number_range() {
    local NUM=$1
    if [ $NUM -ge 0 ] && [ $NUM -le 100 ]; then
        return 0 # success
    else
        return 1 # failure
    fi
}
```

```
# Using returned output
AREA=$(calculate_area 5 10)
echo "Area: $AREA"
```

```
# Using returned status
if check_number_range 50; then
    echo "Number is in range [0,100]"
else
    echo "Number is out of range"
fi
```

- The `source` command (or `.`) executes a script in the **current shell instance**
- Basic usage: `source <script>` or `. <script>`
- Difference from executing a script directly
 - Direct execution (`./script.sh`): Creates a new subshell to run the script
 - Sourcing (`source script.sh`): Executes in the current shell, affecting the current environment
- Effects of sourcing
 - Variables and functions defined in the script become available in the current shell
 - Changes to environment variables persist after the script completes
 - Useful for loading configuration files and environment setup scripts
- Example: `source ~/.bashrc` loads the Bash configuration without creating a new shell

Sourcing vs. Direct Execution

```
# file: config.sh
export MY_VAR="Hello from config"
MY_FUNCTION() { echo "This is a function"; }

# Direct execution (creates subshell)
./config.sh
echo $MY_VAR # MY_VAR is empty (changes lost)

# Sourcing (current shell)
source config.sh
echo $MY_VAR # MY_VAR="Hello from config"
MY_FUNCTION # Function is available
```

- By default a function is executed in the main shell instance
- There are cases where a subshell is spawned
 - When piping is used in the function invocation (e.g.: `my_fn1 | my_fn2`)
 - In case of command substitution (e.g.: `$(my_function)` or ``my_function``)
 - In case of background execution (e.g.: `$(my_function) &` – discussed later)
 - In case of command groups (e.g.: `(my_function)`)

- Loading environment configuration files
 - `source /etc/profile` in login shells
 - `source ~/.bashrc` to reload Bash configuration
- Sharing functions and variables across multiple scripts
 - Create a library script with common functions
 - Source it in other scripts: `source lib.sh`
- Setting up environment variables for a project
 - `source activate.sh` to set project-specific paths
- **NOTE:** Sourced scripts should not contain `exit` statements, as this would terminate the current shell

- To run a command in the background, append `&` at the end of the command.
- Example: `sleep 30 &` will run the sleep command for 30 seconds in the background.
- Background processes allow you to continue using the terminal while the command executes.

- Use the `jobs` command to list all background jobs.
- To bring a background job to the foreground, use `fg` followed by the job number (e.g., `fg %1`).
- To send a background job to the background again, use `bg` followed by the job number (e.g., `bg %1`).
- If no job number is provided, the current job is used
- You can stop a running foreground process by pressing `CTRL+Z`, which suspends the process and returns control to the shell
- For example
`xterm`
`CTRL+Z`
`bg`
- Is equivalent to: `xterm &`

- Signals are software interrupts that provide a way to handle asynchronous events
- A signal is a notification sent to a process to inform it that an event has occurred
- Signals allow processes to respond to events such as user interrupts, termination requests, or errors
- Each signal has a default action (e.g., terminate, ignore, or core dump)
- Processes can define custom signal handlers to catch and handle specific signals
- Signals are identified by numbers and symbolic names (e.g., SIGTERM, SIGKILL)

Number	Name	Default Action	Description
1	SIGHUP	Terminate	Hangup detected (terminal closed)
2	SIGINT	Terminate	Interrupt from keyboard (CTRL+C)
3	SIGQUIT	Core dump	Quit from keyboard (CTRL+\)
9	SIGKILL	Terminate	Kill signal (cannot be caught)
10	SIGUSR1	Terminate	User-defined Signal
15	SIGTERM	Terminate	Termination signal
19	SIGSTOP	Stop	Stop process (cannot be caught)
18	SIGCONT	Continue	Continue stopped process
20	SIGTSTP	Stop	Stop typed at terminal (CTRL+Z)

- The `kill` command sends a signal to a process
- Basic usage: `kill [options] [signal] <PID>`
- Common options
 - `-l`: List all available signals
 - `-s <signal>`: Specify the signal by name
 - `-<signal>`: Specify the signal by number or name (shorthand)
- By default, `kill` sends SIGTERM (signal 15)
- Examples
 - `kill 1234`: Send SIGTERM to process with PID 1234
 - `kill -9 1234`: Send SIGKILL to process with PID 1234 (forceful termination)
 - `kill -SIGTERM 1234`: Send SIGTERM by name
 - `kill -l`: List all available signals
- **NOTE**: SIGKILL cannot be caught or ignored; use it only when necessary

- The `trap` command defines custom signal handlers in shell scripts
- Basic usage: `trap [options] <action> <signal>`
- Allows scripts to catch signals and execute custom code instead of the default action
- Common uses
 - Clean up temporary files when a script is interrupted
 - Perform graceful shutdown operations
 - Log signal events
 - Restore terminal settings
- Examples
 - `trap 'echo "Interrupted!"; exit' SIGINT`: Catch CTRL+C
 - `trap 'cleanup; exit' EXIT`: Execute cleanup on script exit
 - `trap 'rm -f /tmp/tempfile; exit' SIGTERM`: Clean up before termination

Signal Handlers Example

```
#!/bin/bash
# Define cleanup function
cleanup() {
    echo "Cleaning up temporary files..."
    rm -f $TEMP_FILE
    exit 0
}
# Set up signal handlers
trap cleanup SIGINT SIGTERM EXIT
# Create a temporary file
TEMP_FILE="/tmp/script_temp_$$"
touch "$TEMP_FILE"
echo "Temporary file created: $TEMP_FILE"
# Main loop
while true; do
    echo "Press CTRL+C to exit..."
    date >> $TEMP_FILE
    sleep 1
done
```

- Linux uses **Priority (PR)** to determine CPU weight; the lower the number, the higher the priority.
- **Nice Value (NI)**: A user-space value used to influence the kernel's scheduling for normal processes.
 - Range: -20 (Highest) to 19 (Lowest)
 - Default: 0
- **Mapping**: For standard processes, the Kernel Priority is $PR = 120 + NI \in [0, 139]$
 - 120 is the default priority
 - Values < 100 are reserved for real-time processes
 - Some tools (e.g., **top**) normalize the priority in the range $[-100, 39]$
- **Behavior**: Lower nice values are “less nice” to other tasks, receiving a larger proportional share of CPU time.
- **Permissions**: Any user can increase their nice value, only the `root` user can decrease it (set a higher priority).

- **Real-Time Priority:** Used for time-critical tasks (e.g., audio processing, industrial control) that must preempt normal tasks.
- **Priority Range:**
 - RT values range from 1 to 99
 - These always have a higher priority than normal processes
- Scheduling Policies for real-time process
 - `SCHED_FIFO`: First-In, First-Out
 - `SCHED_RR`: Round Robin (time-sliced RT)
- **Risk of starvation:** A runaway RT process can "starve" the rest of the system, including the shell, requiring superuser privileges to manage.

- Non-realtime processes use variable time-slice duration
- Scheduling Policies for normal process
 - Completely Fair Scheduler (CFS)
 - Earliest Eligible Virtual Deadline First (EEVDF) – since 2023
- Rely on variable *vruntime* (time spend on CPU) for scheduling
- Scheduler picks the process with *lowest vruntime* as the next process to schedule
- *Interactive* processes tend to have low vruntime, so they experience *low latency*
- Use red-black tree based on vruntime to implement queue of running processes
 - Tree is auto-balanced: depth remains $\mathcal{O}(\log N)$
 - next process to run is always at the bottom left of the tree

- Use the `ps` command to view process priorities
- Common options
 - `ps aux`: Display all processes with detailed information including NI (nice value)
 - `ps -eo pid,comm,nice`: Display PID, command name, and nice value
 - `ps -eo pid,comm,pri,nice`: Display PID, command, priority (PRI), and nice value
 - Example: `ps -eo pid,pri,nice,comm | tail -n +2 | sort -n -k2 | less`
- Example output columns
 - PID: Process ID
 - PRI: Kernel priority (internal scheduler priority)
 - NI: Nice value (-20 to 19)
 - COMM: Command name

- The `nice` command starts a process with a specified nice value
- Basic usage: `nice [options] <command>`
- Common options
 - `-n <value>`: Set the nice value for the command
 - If no value is specified, the default increment is 10
- Examples
 - `nice -n 10 ./my_script.sh`: Start script with nice value 10 (lower priority)
 - `nice -n -5 ./important_task.sh`: Start script with nice value -5 (higher priority, requires appropriate permissions)
 - `nice ./my_script.sh`: Start script with default nice value 10
- Only the superuser (`root`) can assign negative nice values (higher priority)
- Regular users can only increase the nice value (decrease priority) or keep it the same

- The `renice` command changes the nice value of an already running process
- Basic usage: `renice [options] <new_nice_value> [process_identifier]`
- Common options
 - `-p <pid>`: Specify process by PID (default if no option given)
 - `-g <gid>`: Specify process group by group ID
 - `-u <uid>`: Specify all processes owned by user ID
- Examples
 - `renice 5 -p 1234`: Change process with PID 1234 to nice value 5
 - `renice -10 -p 1234`: Change process with PID 1234 to nice value -10 (requires root)
 - `renice 15 -u username`: Change all processes owned by `username` to nice value 15
- Users can only increase the nice value of their own processes
- Only root can decrease nice values (increase priority)

Process Priority Examples

```
#!/bin/bash
# View processes with nice values
ps aux | head -n5
# View process priority in detail
ps -eo pid,user,cmd,nice,pri | head -n5
# Start a low-priority background task
nice -n 10 sh -c "dd if=/dev/random bs=1M count=10 2>/dev/null | bzip2 > data.bz2" &
# Check the nice value of the process
PID_SH=$!
PID_BZ2=$(pgrep bzip2)
echo "subshell pid is: $PID_SH, bzip2 process pid is $PID_BZ2"
ps -p ${PID_SH},${PID_BZ2} -o pid,comm,nice
# Change the priority of a running process
renice 15 -p $PID_SH
renice 19 -p $PID_BZ2
# View the updated priority
ps -p ${PID_SH},${PID_BZ2} -o pid,comm,nice
rm -f data.bz2
```

- The `cut` command extracts columns or fields from each line of a file or stdin
- Basic usage: `cut [options] <file>`
- Common options
 - `-c <list>`: Extract characters at specified positions (e.g., 1-3, 1,5,10)
 - `-f <list>`: Extract fields (columns) by number
 - `-d <delimiter>`: Specify the field delimiter (default is tab)
 - `-s`: Suppress lines that do not contain the delimiter
- Examples
 - `cut -c 1-5 file.txt`: Extract characters 1-5 from each line
 - `cut -d: -f1 /etc/passwd`: Extract usernames (first field) from `passwd` file
 - `cut -d, -f2,4 data.csv`: Extract 2nd and 4th fields from CSV file
 - `ps aux | cut -c 1-80`: Extract first 80 characters from `ps` output

- The **grep** command searches for lines matching a pattern in files or stdin
- Basic usage: **grep [options] <pattern> <file>**
- The pattern can be a literal string or a regular expression
- If no file is provided recursive searches examine the working directory, and non-recursive searches read standard input
- Common options
 - **-i**: Case-insensitive search
 - **-v**: Invert match (display lines that do NOT match)
 - **-n**: Display line numbers along with matching lines
 - **-c**: Count the number of matching lines
 - **-l**: Display only filenames containing matches
 - **-r**: Recursively search directories
 - **-e**: Use regular expressions (discussed later)
 - **-E, -G, -P**: Use {extended|regular|perl} regular expressions (discussed later)
 - **-F**: Treat pattern as literal string (no regex interpretation)

- `-w`: Match whole words only (not partial matches)
- `-x`: Match entire lines only
- `-o`: Display only the matching part of each line
- `-A <num>`: Display <num> lines after each match (context after)
- `-B <num>`: Display <num> lines before each match (context before)
- `-C <num>`: Display <num> lines before and after each match
- `--color`: Highlight matching text in color
- `-m <num>`: Stop after finding <num> matches

- `grep "error" log.txt`: Find lines containing “error”
- `grep -i "warning" log.txt`: Case-insensitive search for “warning”
- `grep -c "success" log.txt`: Count lines containing “success”
- `grep -c "success" *.txt |grep -v ":0"`: remove files without the word “success”
- `grep -w "test" file.txt`: Match “test” as a whole word, not “testing”
- `grep -B2 -A2 "ERROR" log.txt`: 2 lines of context before and after “ERROR”
- `grep -C2 "ERROR" log.txt`: same result
- `grep -o "[:digit:]*" data.txt`: Display only numbers from each line
- `grep -r "function" src/`: Search for “function” in all files under src/ directory

Lab experience

- Create a script with at least the following functions
 - `check_process_load`: prints the cpu load using the `/proc/loadavg` metric on a timescale of 1 min
 - `check_cpu_load`: prints the cpu utilization using the `/proc/stat` file
 - `check_memory_usage`: prints the amount of memory used by processes and the total amount of memory installed on the system (use `/proc/meminfo`)
- The main script should call these functions sequentially.
- The function outputs should be captured using command substitution to provide a concise system summary

Structure of the /proc/loadavg file

- Fields 1-3: Average CPU load over the last 1/5/15 minutes
- Field 4: Number of currently running processes / Total number of processes
- Field 5: PID of the last process that was launched

```
get_process_load() {  
    local load_1min=$(cat /proc/loadavg | cut -d' ' -f1)  
    echo "$load_1min"  
}
```

Structure of the `/proc/stat` file

- several lines (CPU global statistics, per-core stats, interrupt, context switches, etc..)
- Each CPU contains the following fields: `user nice system idle iowait irq softirq steal guest guest_nice`
 - `user, nice`: Time (in jiffies) spent in user mode with normal / low priority (nice)
 - `system`: Time spent in kernel mode
 - `idle`: Time spent idle (doing nothing)
 - `iowait`: time spent waiting for I/O
 - `irq, softirq` time spent servicing interrupts ()
 - `steal, guest, guest_nice`: relevant only for virtualization

- All metrics in `/proc/stat` are cumulative from the system startup
- Must obtain data in the last period
- Use global variables to store old values
- Using `LANG` variable to force decimal separator

```
((old_total=0))
((old_idle=0))
get_cpu_load() {
    local cpu_line=$(grep 'cpu ' /proc/stat)
    read -r label user nice system idle iowait irq softirq steal guest gnice<<< "
        $cpu_line"
    local total=$((user + nice + system + idle + iowait + irq + softirq + steal +
        guest + guest_nice))
    local cpu_used_perc=$(echo "scale=2; 100 * (($total-$old_total)-($idle-$old_idle
        )) / ($total-$old_total)" | bc)
    ((old_total=total))
    ((old_idle=idle))
    LANG=C printf "%.2f%" $cpu_used_perc
}
```

Structure of the `/proc/meminfo` file (only main fields)

- **MemTotal**: Total usable RAM in kilobytes
- **MemFree**: Free memory that is immediately available without reclaiming
- **MemAvailable**: Estimate of available memory for starting new applications
- **Buffers**: Memory used by the kernel for temporary storage
- **Cached**: Memory used for page cache and file-backed pages
- **SwapTotal**: Total swap space available in kilobytes
- **SwapFree**: Swap space that is currently free

```
get_memory_usage() {  
    local total_kb=$(grep "MemTotal" /proc/meminfo | tr -s ' ' | cut -d' ' -f2)  
    local free_kb=$(grep "MemFree" /proc/meminfo | tr -s ' ' | cut -d' ' -f2)  
    local usage_perc=$(echo "scale=2; (($total_kb - $free_kb) / $total_kb) * 100" |  
        bc)  
    local used_mb=$(echo "scale=2; ($total_kb - $free_kb) / 1024" | bc)  
    local total_mb=$(echo "scale=2; $total_kb / 1024" | bc)  
    echo "${used_mb}MB/${total_mb}MB (${usage_perc}%)"  
}
```

- Forever loop
- Using `true` command
- One second for each iteration
- Cannot execute in subshell functions working with global variables
 - Can it be improved?

```
#!/bin/bash
source get_process_load.sh
source get_cpu_load.sh
source get_memory_usage.sh
while true
do
    echo -n "Load (1m):" $(get_process_load) "| CPU usage: "
    get_cpu_load
    echo " | Memory: " $(get_memory_usage)
    sleep 1
done
```

```
#!/bin/bash
PID=$1
if [[ -z "$PID" ]]; then
    echo "Usage: $0 <PID>"; exit 1; fi
if [[ -d "/proc/$PID" ]]; then
    PROC_NAME=$(ps -p "$PID" -o comm=)
    echo "Process $PID ($PROC_NAME) is currently running."
    read -p "Do you want to terminate this process? (y/n): " CONFIRM
    case "$CONFIRM" in
        [yY] | [yY][eE][sS])
            echo "Attempting to kill $PID..."
            kill "$PID" 2>/dev/null
            if [[ $? -eq 0 ]]; then echo "Success: Process $PID terminated."
            else echo "Error: Failed to kill $PID. (Check permissions)"; fi
            ;;
        [nN] | [nN][oO]) echo "Operation aborted." ;;
        *) echo "Invalid input. No action taken." ;;
    esac
else echo "Error: PID $PID not found in /proc."; fi
```

- The Priority Guardian
 - The script should identify a process by name, check its current *niceness* value, and offers to increase its priority (lower the nice value).
 - Create a function `get_niceness()` that takes a PID and returns its current priority.
 - In the main script, take a process name (e.g., `firefox`) as input, gets its PID (there can be more than one) and, if the PID exists, calls `get_niceness`
 - Use a case statement to ask the user: "High (-10)", "Low (10)", or "No Change".
 - Execute `renice` based on the choice and report success using \$?
- Centered Rectangle
 - Create a function `draw_rect()` that accepts 4 parameter: `x1`, `y1`, `x2`, `y2` and draws a rectangle (using the character `+`) using these coordinates
 - Create a function `rect()` that computes the variables `x1`, `y1`, `x2`, `y2` such that the rectangle is centered on the screen and has half the screen size
 - Intercepts the signal `SIG_WINCH` to clear the screen and invoke `rect` when the terminal is resized
 - Waits forever

- The CPU frequency monitor
 - Create a function that reads the CPU frequency in `/sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq` and converts the value from Hz to GHz
 - Keep track of old CPU frequency and evaluate in a 1 sec loop if frequency is increasing or decreasing
 - Check also if the CPU frequency is the highest or lowest frequency (in `.../cpufreq/cpuinfo_max_freq` and `.../cpufreq/cpuinfo_min_freq`).
 - Print at every iteration the current frequency, if it is within 1% from the maximum and minimum frequencies
 - Print if the frequency is increasing, decreasing or remaining stable (within 10% from the last value)
 - Warning: bash math supports only integers

Bonus tips and tricks

- Discuss the meaning of the following code:
`:(){ :|:& };;:`
- Function definition `:(){ }`: defines a function named `:`
- Body of the function `:|:&`
 - Executes two instances of the function in background
 - The pipe sign (`|`) is just to make two instances of the function in concurrent execution
 - No data is exchanged through the pipe
 - The couple of functions is executed in background: the function can go on `&`
- First function invocation `:`
 - The first instance of the function is called
 - The function invocation is separated from the function definition by a semicolon `;`

- A Here Document (heredoc) is a way to pass multi-line input to a command

- Basic syntax:

```
command << DELIMITER  
line 1  
line 2  
...  
DELIMITER
```

- The DELIMITER marks the beginning and end of the document
- Common delimiters: EOF, END, INPUT
- The heredoc content is treated as standard input to the command
- Useful for creating multi-line strings, sending email, or providing input to interactive programs

- By default, variables and command substitutions are expanded within a heredoc
- To prevent expansion, quote the delimiter:
`command << 'DELIMITER'`
- Examples
 - With expansion: `cat << EOF` expands `$VAR`
 - Without expansion: `cat << 'EOF'` preserves literal `$VAR`
- Indentation can be preserved using `<<-`:
`command <<- DELIMITER`
- This allows the closing delimiter to be indented for readability

Here Documents Examples

```
#!/bin/bash
# Simple heredoc sending to cat
cat << EOF
This is a here document.
It can span multiple lines.
EOF
# Heredoc with variable expansion
NAME="Alice"
cat << EOF
Hello, $NAME!
Today is $(date +%A)
EOF
```

```
# Heredoc without variable expansion
cat << 'EOF'
The variable $NAME is not expanded.
Command substitution $(date) is not
    executed.
EOF
# Heredoc sent to a file
cat > script_output.txt << EOF
Generated content at $(date)
Created for user: $USER
EOF
```

Here Documents Examples

```
#!/bin/bash
# Indented heredoc with <<-
cat <<- EOF
    This is indented in the script
    but the output is not indented
EOF
# Heredoc with command input
mysql -u root -p << EOF
USE mydatabase;
SELECT * FROM users;
EOF
```

```
# Heredoc in variable assignment
MULTILINE=$(cat << 'EOF'
Line 1
Line 2
Line 3
EOF
)
echo "$MULTILINE"
```

- `getopts` is a Bash built-in command for parsing command-line options
- Simplifies the process of handling options in scripts
- Basic syntax
`getopts <optstring> <variable> [<args>]`
- Parameters
 - `optstring`: String specifying valid options (e.g., `abc`, `a:b::c`)
 - `variable`: Variable to store the current option
 - `args`: Optional list of arguments to parse (default is command-line arguments)
- Option format in `optstring`
 - Single letter: Option without argument (e.g., `a`)
 - Followed by colon: Option requires an argument (e.g., `a:`)
 - Followed by two colons: Option argument is optional (e.g., `a::`)

- **OPTIND**: Index of the next argument to be processed
 - Automatically incremented by **getopts**
 - Can be reset to 1 to reparse options
 - After **getopts** completes, OPTIND points to the first non-option argument
- **OPTARG**: Contains the argument for the current option
 - Only set when the option requires an argument
 - Empty if the option does not take an argument
- **OPTERR**: Controls error messages (default is 1)
 - Set OPTERR=0 to suppress error messages for invalid options

Basic getopt Example

```
#!/bin/bash
# Parse options: -a and -b with argument, -c with optional argument
while getopt "a:b:c::" opt; do
    case $opt in
        a) echo "Option -a with argument: $OPTARG"
           ;;
        b) echo "Option -b with argument: $OPTARG"
           ;;
        c) echo "Option -c with optional argument: $OPTARG"
           ;;
        \?) echo "Invalid option: -$OPTARG" >&2
            exit 1
           ;;
    esac
done
# Get remaining positional arguments
shift $((OPTIND - 1))
echo "Remaining arguments: $@"
```

getopts Example: Script Usage

```
$ ./script.sh -a value1 -b value2 file1 file2  
Option -a with argument: value1  
Option -b with argument: value2  
Remaining arguments: file1 file2
```

```
$ ./script.sh -c value3 -a value1  
Option -c with optional argument: value3  
Option -a with argument: value1  
Remaining arguments:
```

```
$ ./script.sh -c -a value1 file3  
Option -c with optional argument:  
Option -a with argument: value1  
Remaining arguments: file3
```

Advanced getopt Example

```
#!/bin/bash
# Script with multiple options and help
usage() {
    echo "Usage: $0 [-h] [-v] [-f FILE] [-n
    NUMBER]"
    echo " -h Display this help message"
    echo " -v Verbose mode"
    echo " -f FILE Specify input file"
    echo " -n NUMBER Specify a number"
    exit 0
}
VERBOSE=0
FILE=""
NUMBER=""
while getopt "hvf:n:" opt; do
    case $opt in
        h) usage
            ;;
        v) VERBOSE=1
            ;;
    esac
done
```

```
f) FILE="$OPTARG"
    ;;
n) NUMBER="$OPTARG"
    ;;
\?) echo "Invalid option: -$OPTARG"
    >&2
    usage
    ;;
:) echo "Option -$OPTARG requires an
    argument" >&2
    exit 1
    ;;
done
# Display parsed options
[ $VERBOSE -eq 1 ] && echo "Verbose mode"
[ -n "$FILE" ] && echo "Input file: $FILE"
[ -n "$NUMBER" ] && echo "Number: $NUMBER"
```

- \?) case: Triggered when an invalid option is provided
- :) case: Triggered when a required argument is missing
- Default error messages can be suppressed by setting `OPTERR=0`
 - Useful for custom error handling and messages
 - Allows complete control over error reporting
- Example of suppressing default errors:
`getopts "a:b:" opt 2>/dev/null`

Manual Parsing

```
if [ "$1" = "-a" ]; then
    OPTION_A="$2"
    shift 2
elif [ "$1" = "-b" ]; then
    OPTION_B="$2"
    shift 2
fi
# Complex and error-prone
```

Using getopts

```
while getopts "a:b:" opt; do
    case $opt in
        a) OPTION_A="$OPTARG"
           ;;
        b) OPTION_B="$OPTARG"
           ;;
    esac
done
# Clean, reliable, POSIX-compliant
```

- Always use `getopts` instead of manual parsing
 - More reliable and maintainable
 - Handles edge cases automatically
 - POSIX-compliant
- Use `\?)` and `:)` cases for error handling
- Provide a `-h` or `--help` option for usage information
- Document all available options in help text
- Use `shift $((OPTIND - 1))` to remove parsed options before processing remaining arguments
- Consider using `OPTERR=0` for complete error control
- Test with various option combinations and edge cases